

Checkpoint de Velocidade

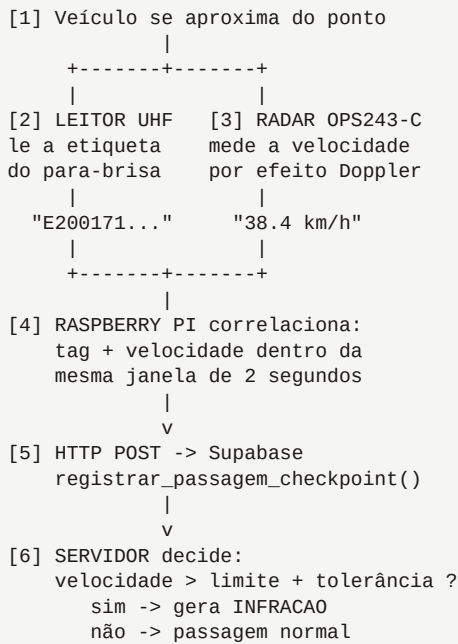
Guia de integração do Raspberry Pi com o radar Doppler e o leitor de tags RFID UHF

Destino	Equipe de campo / TI
Hardware	Raspberry Pi 4 · OmniPreSense OPS243-C · Leitor UHF
Endpoint	dadosfrota.apicesystem.shop
Pre-requisito	Migration 20260818000007 aplicada no banco

Como o conjunto funciona

Leia esta página antes de encostar no hardware. Entender o fluxo evita 90% dos erros de montagem.

O Raspberry Pi é o cérebro do checkpoint. Ele não decide se houve infração — quem decide isso é o banco de dados. O papel do Pi é apenas: **ler dois sensores, juntar as duas leituras no mesmo veículo e enviar para o sistema**. Toda a regra de limite, tolerância e gravidade já esta implementada no servidor.



Fluxo completo. O Pi só executa os passos 4 e 5.

Por que a decisão fica no servidor

Se o limite mudar (obra alterou a sinalização de 40 para 30 km/h), você muda num campo da tela e todos os checkpoints passam a valer o novo limite na hora. Não precisa subir no poste para reprogramar nada.

O que você vai montar

Equipamento	Conexão com o Pi	Como o Pi enxerga
Radar OPS243-C	Cabo USB (o próprio radar tem porta USB)	/dev/ttyACM0
Leitor Control iD iDUHF	Cabo de rede + fonte própria de 12V/2A	ele chama o Pi
Câmera IP (opcional)	Cabo de rede	IP na rede local
Modem 4G	USB ou cabo de rede	interface de rede

Os dois sensores se conectam de formas opostas

O **radar** é USB: o Pi abre a porta e lê. O **leitor de tags** é rede, e o sentido se inverte — é ele quem chama o Pi a cada leitura, num endereço que você configura na tela dele. Por isso o Pi precisa de IP fixo, e por isso o script sobe um pequeno servidor esperando essas chamadas.

1 Preparar o Raspberry Pi

Sistema operacional, acesso remoto e bibliotecas. Faça na bancada, com monitor e teclado, antes de levar para o campo.

1.1 Gravar o sistema no cartão

Use o **Raspberry Pi Imager** no seu computador. Escolha **Raspberry Pi OS Lite (64-bit)** — a versão sem interface gráfica, porque o Pi vai ficar num poste sem monitor e o sistema leve consome menos energia da bateria.

Ainda no Imager, clique na engrenagem e já deixe configurado:

- Nome do dispositivo: **checkpoint-01** (use o número do ponto)
- Ativar SSH com usuário e senha — é assim que você vai acessar no campo
- Configurar a rede Wi-Fi ou deixar cabeado, conforme o local
- Fuso horário: **America/Fortaleza** ou o da sua obra

1.2 Primeiro acesso e atualização

```
ssh pi@checkpoint-01.local

sudo apt update && sudo apt full-upgrade -y
sudo apt install -y python3-pip python3-venv git
```

1.3 Criar o ambiente do projeto

O ambiente virtual isola as bibliotecas do checkpoint do resto do sistema. Isso evita que uma atualização do sistema quebre o script.

```
mkdir -p ~/checkpoint && cd ~/checkpoint
python3 -m venv .venv
source .venv/bin/activate

pip install pyserial requests
```

1.4 Liberar o acesso a porta serial

Por padrão o usuário não pode ler a porta USB do radar. Este comando resolve — é obrigatório reiniciar depois.

```
sudo usermod -aG dialout $USER
sudo reboot
```

Ponto de verificação

Ao final desta etapa você deve conseguir entrar no Pi por SSH e o comando `groups` deve listar `dialout`. Se não listar, o radar não vai funcionar na Etapa 3.

2 Cadastrar o checkpoint e pegar o token

Faca esta etapa no sistema, pelo navegador. O token é a senha do dispositivo — sem ele o Pi não consegue enviar nada.

2.1 Criar o ponto no sistema

- Acesse **SMS / SSMA** → **Controle de Velocidade**
- Abra a aba **Checkpoints** e clique em **Novo checkpoint**
- Preencha a obra, o nome do ponto (ex.: *Acesso AG-14*) e o limite da via
- Informe a **tolerância** — sugerimos começar com 5 km/h
- Se souber, preencha latitude e longitude para o ponto aparecer no mapa

O que a tolerância faz

Ela absorve a margem de erro do radar. Num ponto de 40 km/h com tolerância 5, o sistema só registra infração acima de **45 km/h**. Isso evita autuar motorista por causa de imprecisão do equipamento — o que geraria contestação e desgaste com a equipe.

2.2 Copiar o token do dispositivo

Depois de criar, o checkpoint aparece na lista. Clique em **Copiar token** no card. Você vai colar esse valor no arquivo de configuração do Pi na Etapa 5.

Trate o token como senha

Quem tiver o token consegue registrar passagens falsas naquele ponto. Não coloque em grupo de WhatsApp, não deixe escrito na caixa do equipamento e não suba para o GitHub. Se vazar, basta desativar o checkpoint na tela e criar outro.

2.3 Vincular as tags aos veículos

Cada veículo da frota precisa de uma etiqueta UHF colada no para-brisa e registrada no sistema. Sem isso o checkpoint registra a passagem, mas com o veículo marcado como *não identificado*.

- Na mesma tela, abra a aba **Tags RFID** e clique em **Vincular tag**
- Selecione o veículo e cole o **EPC** da etiqueta
- O EPC você obtém aproximando a etiqueta do leitor — veja a Etapa 4

Cole a etiqueta sempre na mesma posição em todos os veículos (sugestão: canto superior do para-brisa, atrás do retrovisor). Posição consistente significa alcance de leitura consistente.

3 Conectar e testar o radar

O OPS243-C é um sensor Doppler que fala por texto simples pela porta USB. Você vai configurar e ver a velocidade na tela antes de escrever qualquer código.

3.1 Ligar e identificar a porta

Conecte o radar numa porta USB do Pi e rode:

```
ls /dev/ttyACM*  
# Saída esperada: /dev/ttyACM0
```

Se não aparecer nada, o cabo é o primeiro suspeito — e o mais comum. Muito cabo de celular tem só os fios de energia, sem os de dados: o LED do radar acende normalmente e o Pi nunca fica sabendo que ele existe. Antes de procurar defeito, troque por um cabo que você sabe que transfere dados.

O baud rate não importa

O OPS243 é USB CDC puro: a comunicação acontece pelo protocolo USB e o baud é ignorado. Testamos 9600, 19200, 38400, 57600, 115200 e 230400 — saída idêntica em todos. Se algum tutorial insistir num valor específico, pode ignorar; qualquer um funciona.

3.2 Silenciar a distância antes de tudo

O OPS243-C mede velocidade e distância. De fábrica ele reporta as duas, e a distância domina a porta: numa medição de bancada, cerca de 90% das linhas eram distância de um alvo parado. Pior, esse volume **engole a resposta dos outros comandos** — foi o que fez a identificação do módulo parecer não funcionar.

Então a ordem importa: **silencie a distância primeiro**, depois converse com o radar.

3.3 Comandos que o radar entende

Comando	O que faz
Od	Desliga o reporte de distância. Sempre o primeiro.
OS	Liga o reporte de velocidade
OM	Liga a magnitude do sinal junto de cada leitura
UK	Passa a reportar em quilômetros por hora
M>N	Ignora ecos com magnitude abaixo de N
??	Modelo, versão de firmware e configuração atual
A!	Grava a configuração na memória do radar

A convenção é maiúscula liga, minúscula desliga

OS liga o reporte de velocidade, Os desliga. Vale para todos os comandos O?. E cuidado com as letras: W é WaitTime e B é BlanksPref — não são WiFi nem Bluetooth. Comando inválido devolve `illegal value`, então dá para sondar sem medo.

3.4 Configurar e ver ao vivo

Rode o teste abaixo e passe a mão na frente do radar:

```
python3 - <<'FIM'
import serial, time
r = serial.Serial("/dev/ttyACM0", 115200, timeout=1)
time.sleep(0.5)
for cmd in (b"Od\n", b"OS\n", b"OM\n", b"UK\n"):
    r.write(cmd); time.sleep(0.4)
    print("cfg:", r.read(400).decode("ascii", "replace").strip()[:70])

print("Passe a mao na frente (Ctrl+C para sair)")
while True:
    linha = r.readline().decode("ascii", "replace").strip()
    if linha:
        print("    ", linha)
FIM
```

3.5 Entender o que o radar devolve

A saída **não é JSON**, como parte da documentação sugere. É uma linha por leitura, no formato unidade seguida de valor — e com OM ligado entra um campo a mais:

```
"kmph",171,2.6      <- com OM ligado
|      |      |
|      |      +-- velocidade em km/h (sinal indica direcao)
|      +----- magnitude do eco: a forza do sinal
+----- unidade

"kmph",2.6          <- sem OM, so dois campos
"m",2.0             <- distancia (some depois do Od)
{"SpeedUnit":"kmph"} <- resposta de comando, nao e leitura
```

A velocidade é sempre o **ULTIMO** campo. Qualquer parser precisa aceitar as duas formas, com e sem magnitude.

A magnitude é o que separa veículo de fantasma

Numa medição de bancada, o ruído de fundo apareceu com magnitude **1** e velocidades caóticas — de menos 110 a mais 98 km/h, sem sentido físico. O alvo real veio com magnitude entre **2 e 589** e velocidade coerente. A separação é limpa, e é por isso que vale ligar o OM: sem magnitude não há como distinguir os dois, porque o ruído cai bem dentro da faixa de velocidade que você quer autuar.

3.6 Calibrar o corte de magnitude

Este é o ajuste que decide se o checkpoint vai funcionar ou gerar infração contra ninguém. E ele é **específico de cada posição** — o valor que serve na bancada não serve no poste.

Com o radar já montado na posição definitiva, rode:

```
python3 - <<'FIM'
import serial, time
s = serial.Serial("/dev/ttyACM0", 115200, timeout=1); time.sleep(0.5)
for c in (b"Od\n", b"OS\n", b"OM\n", b"UK\n"):
    s.write(c); time.sleep(0.6); s.read(3000)

print(">>> VIA VAZIA. Nao passe nada na frente por 60 segundos. <<<")
time.sleep(3); s.reset_input_buffer()
mags, fim = [], time.time() + 60
while time.time() < fim:
    l = s.readline().decode("ascii", "replace").strip()
    p = l.split(",")
    if len(p) >= 3:
        try: mags.append(float(p[1]))
        except ValueError: pass
s.close()

print(f"{len(mags)} leituras de ruído em 60s")
if not mags:
    print("  silencio total - posicao excelente. Use MAGNITUDE_MIN=5")
else:
    mags.sort()
    p50 = mags[len(mags) // 2]
    p95 = mags[min(int(len(mags) * 0.95), len(mags) - 1)]
    print(f"  mediana {p50:.0f}  p95 {p95:.0f}  maximo {max(mags):.0f}")
    print(f"  >>> use MAGNITUDE_MIN={int(p95) + 3}")
    if max(mags) > p95 * 3:
        print("    (o maximo destoa do p95: algo passou na frente)")
        print("    durante o teste. O p95 ignora esse pico.")
FIM
```

Use o valor sugerido no .env da Etapa 5. O calculo usa o percentil 95 e nao o maximo: assim um unico pico — alguém que passou, um passaro — nao contamina o corte e faz o filtro barrar veiculo de verdade.

Se o ruído não parar, mexa na posição antes de mexer no filtro

Ruído alto costuma ser o entorno, não o equipamento. Em bancada tivemos leituras fantasma travadas em 38 km/h que sumiram por completo depois de reposicionar a placa. Verifique: metal a menos de um metro à frente, cabo encostando nos patches, radar apontado para o teto ou para um barranco, e qualquer coisa girando no campo de visão — ventilador, gerador, compressor, exaustor de contêiner. Pá de ventilador faz uns 36 km/h na ponta e o radar a lê como veículo, 24 horas por dia.

3.7 Não grave com A!

A configuração do radar **não persiste**: a cada vez que ele é religado, volta ao padrão de fábrica. O comando A! grava na memória e resolveria isso — mas a recomendação aqui é **não usar**.

O script reenvia todos os comandos a cada conexão. Com isso a configuração fica no código, versionada, e vale para qualquer radar: se um módulo queimar e você plugar outro, ele já sobe configurado igual. Gravado na flash, o ajuste vira estado invisível daquele equipamento específico, e daqui a seis meses ninguém lembra o que está lá dentro.

4 Conectar o leitor de tags

Control iD iDUHF. Diferente do radar, ele não se liga ao Pi por cabo — os dois conversam pela rede, e é o leitor quem procura o Pi.

4.1 O que vem na caixa e o que falta comprar

Item	Situação
Antena iDUHF 420 × 420 × 60 mm · 2,27 kg	Inclusa
MAE — Módulo de Acionamento Externo	Opcional, só se houver portão a acionar
Kit de instalação (apoio + abraçadeira)	Incluso
Fonte 12V / 2A	Vendida à parte — o manual exige fonte sem ruído
Mastro de suporte	Vendido à parte
Chave fixa de 13 mm	Vendida à parte

Ele já é IP65 — não precisa de caixa

Diferente do Raspberry Pi e do radar, o iDUHF foi feito para ficar exposto. A caixa hermética da seção de instalação é só para o Pi. O que o iDUHF pede é vedação correta dos prensa-cabos: passe os cabos **desencapados** pelos furos e só depois crimpe os conectores — cabo já crimpado não passa.

4.2 Alimentação separada

O leitor **não** é alimentado pelo Pi. Ele precisa de fonte própria de 12V/2A, e o manual do fabricante insiste em fonte de qualidade e sem ruído. Vale levar a sério: ruído de alimentação é candidato recorrente a leitura errática em equipamento de RF.

4.3 Endereço fixo para o Pi

No modo que vamos usar, é o **leitor que chama o Pi** a cada leitura. Se o Pi trocar de IP num reinício, o leitor passa a falar sozinho — e o sintoma é silêncio, sem erro em lugar nenhum.

Antes de configurar, reserve um IP fixo para o Pi no roteador (por endereço MAC) ou configure IP estático nele. Anote o endereço:

```
hostname -I | awk '{print $1}'
```

4.4 Configurar o leitor

O iDUHF tem software web embarcado. Descubra o IP dele e acesse pelo navegador, com usuário e senha admin:

```
ip neigh | grep -v FAILED
# se nao aparecer:
sudo apt install -y arp-scan && sudo arp-scan --localnet
```


Na interface, vá em **Configurações** → **Modo de Operação** → **Conectividade** e preencha o bloco **Configurar modo monitor**:

Campo	Valor
Hostname	o IP fixo do Pi
Porta	8000 o mesmo valor de LEITOR_PORTA no .env
Request timeout	5000
Caminho do endpoint	api/notifications
Ativar modo online	deixe em Offline

Por que Offline, se queremos o leitor conectado

O nome engana. O toggle *modo online* faz o leitor **perguntar e esperar** a resposta do Pi a cada leitura, antes de decidir. Num poste, isso transforma uma falha do Raspberry em falha do equipamento de campo. O **modo monitor**, que é o que configuramos acima, avisa de forma assíncrona: se o Pi cair, o leitor continua operando e só perdemos o registro daquele intervalo. E o código da tag vem nos dois modos, então não há nada a ganhar com o acoplamento mais forte.

4.5 Conferir o que o leitor envia

Com o script rodando e `DEBUG=1` no `.env`, aproxime uma tag. Deve aparecer:

```
POST /api/notifications/dao
{"object_changes": [{"object": "access_logs", "type": "inserted",
  "values": {"card_value": "223338326031", "user_id": "",
    "event": "3", "portal_id": "1", ...}}]}

POST /api/notifications/device_is_alive
{"access_logs": 3, "device_id": 4609032484573845, "time": ...}

-- no log do script, com DEBUG=1 --
20:31:44   tag: 223338326031
```

O primeiro é o evento da tag; o segundo é o heartbeat, que o leitor manda sozinho de tempos em tempos para dizer que esta vivo.

4.6 O código da etiqueta não é o que o leitor reporta

Este é o detalhe que mais custa tempo se passar despercebido. A etiqueta traz três representações do mesmo número:

na etiqueta	HEX: 34680F	
	DEC: 3434511	
	WG : 052,26639	
o leitor reporta	223338326031	= 0x34 0000 680F
		^ ^ ^ ^ ^ ^
		facility cartao
regra confirmada:	(facility << 32) cartao	

Os tres campos da etiqueta sao o mesmo numero. Nenhum deles é o valor que o leitor reporta.

O leitor insere quatro zeros entre o *facility code* e o número do cartão. Cadastrar o que está impresso na etiqueta faria **toda passagem cair como “veículo não identificado”**, sem erro no log e sem nada apontando a causa.

A tela já resolve isso

Em **Tags RFID** → **Vincular tag**, escolha o formato que você está lendo na etiqueta (HEX, DEC ou WG) e digite o código como está. O sistema converte e mostra o valor final antes de salvar — confira esse preview contra o que aparece no log do script. Se divergir, algo mudou na codificação do lote de tags.

4.7 Alinhar o leitor com o radar

O feixe do iDUHF abre **30°** (15° para cada lado) e alcança até 15 m, dependendo da tag e da instalação. O radar tem seu próprio alcance, medido na Etapa 3.

Os dois precisam cobrir **o mesmo trecho da via**. Se a antena lê a etiqueta vinte metros antes de o radar medir a velocidade, as duas leituras não caem na mesma janela de correlação — e o sistema registra velocidade sem saber de qual veículo.

Ajuste	Efeito	Como calibrar
Potência da antena	Mais alcance, mas lê veículo da pista ao lado	Comece baixo e suba até ler com folga só na sua faixa
Ângulo	Define onde o veículo é detectado	Aponte para o mesmo trecho que o radar cobre
Altura	A etiqueta fica a cerca de 1,2 m do solo, no para-brisa	Entre 2 e 3 m, levemente inclinada para baixo

O manual do fabricante também alerta: **não instale duas unidades do iDUHF cobrindo a mesma região de leitura**. Relevante se você planeja checkpoints próximos um do outro.

4.8 Colar as etiquetas

Cole sempre na mesma posição em todos os veículos — sugestão: canto superior do para-brisa, atrás do retrovisor. Posição consistente significa alcance de leitura consistente, o que reduz a variação entre um veículo e outro e facilita calibrar a potência uma vez só.

5 Instalar o script de integração

O programa que junta as duas leituras e envia ao sistema. Copie os dois arquivos abaixo para a pasta ~/checkpoint no Pi.

5.1 Arquivo de configuração

Crie o arquivo ~/checkpoint/.env com os dados do seu ponto. Este arquivo guarda o token, então proteja o acesso a ele.

```
SUPABASE_URL=https://dadosfrota.apicesystem.shop
SUPABASE_ANON_KEY=<a chave anon do sistema>
DEVICE_TOKEN=<cole aqui o token copiado na Etapa 2>

RADAR_PORT=/dev/ttyACM0
RADAR_BAUD=115200

# Porta em que o Pi escuta o leitor UHF. Mesmo valor do campo
# "Porta" no modo monitor, na tela do iDUHF. 0 desativa.
LEITOR_PORTA=8000

# Velocidade abaixo desta e ignorada (km/h)
VELOCIDADE_MIN=5.0
# Corte de magnitude medido na Etapa 3.6. Filtra o ruído de fundo.
MAGNITUDE_MIN=5
# Janela para casar a tag RFID com a velocidade (segundos)
JANELA_S=2.0
# Fecha a passagem por tempo, não só por silêncio. Protege contra
# ruído contínuo, que senão deixaria a passagem aberta para sempre.
DURACAO_MAX_S=8.0
# DEBUG=1 imprime cada leitura e diz qual filtro a descartou.
# Ligue para diagnosticar, desligue em operação normal.
DEBUG=0
```

```
chmod 600 ~/checkpoint/.env
```

Restringe a leitura do arquivo ao seu usuário.

5.2 O programa principal

Salve como ~/checkpoint/checkpoint.py. O código está comentado para você conseguir ajustar depois.

```
#!/usr/bin/env python3
"""
checkpoint.py - Integração radar OPS243-C + leitor UHF -> Sistema Apice

Arquitetura: duas threads independentes alimentam filas com carimbo de
tempo. O correlacionador junta velocidade + tag da mesma passagem e envia
uma única linha ao servidor. O servidor decide se houve infração.
"""
import json, os, queue, threading, time
from datetime import datetime, timezone
import requests, serial

# --- Configuração (lida do arquivo .env) -----
def carregar_env(caminho=".env"):
    if os.path.exists(caminho):
        for ln in open(caminho):
            ln = ln.strip()
            if ln and not ln.startswith("#") and "=" in ln:
                k, v = ln.split("=", 1)
                os.environ.setdefault(k.strip(), v.strip())

carregar_env(os.path.join(os.path.dirname(__file__), ".env"))

URL = os.environ["SUPABASE_URL"]
ANON_KEY = os.environ["SUPABASE_ANON_KEY"]
TOKEN = os.environ["DEVICE_TOKEN"]
PORTA = os.getenv("RADAR_PORT", "/dev/ttyACM0")
BAUD = int(os.getenv("RADAR_BAUD", "19200"))
# Porta em que o Pi escuta as notificacoes do leitor UHF (Control id iDUHF).
# O mesmo valor vai no campo "Porta" do modo monitor, na tela do leitor.
# Vazio ou 0 desativa o leitor: so a velocidade e registrada.
LEITOR_PORTA = int(os.getenv("LEITOR_PORTA", "0") or 0)

# Janela de correlação: tag e velocidade dentro deste intervalo
# são consideradas o mesmo veículo
JANELA = float(os.getenv("JANELA_S", "2.0"))
# Abaixo disso e ruído (pessoa andando, galho ao vento)
VEL_MIN = float(os.getenv("VELOCIDADE_MIN", "5.0"))
# Magnitude minima do eco para considerar veiculo. Calibrar no local:
# rodar com a via vazia e ver ate quanto o ruído chega. 0 desativa.
MAG_MIN = float(os.getenv("MAGNITUDE_MIN", "0"))
# Silêncio que encerra uma passagem
FIM_PASS = 1.2
# Teto de duracao de uma passagem. Sem isso, ruído continuo mantem a
# passagem sempre "aberta" (nunca ha silencio) e NADA e enviado -- o
# script fica mudo para sempre. Um veiculo cruza o ponto em poucos
# segundos; passou disso, fecha e envia o que tem.
DUR_MAX = float(os.getenv("DURACAO_MAX_S", "8.0"))
# DEBUG=1 no .env imprime cada leitura crua do radar. Essencial para
# diagnosticar em campo por que uma passagem nao foi registrada.
DEBUG = os.getenv("DEBUG", "0") not in ("0", "", "false", "False")

# Contadores para o heartbeat (sem eles o script fica mudo e nao da
# para saber se esta vivo, se o radar emite, ou se o filtro descarta)
stats = {"linhas": 0, "leituras": 0, "descartadas": 0, "passagens": 0,
         "tags": 0, "leitor_visto_em": 0.0}
```

continua na próxima página — parte 1 de 7

```

RPC = f"{URL}/rest/v1/rpc/registrar_passagem_checkpoint"
CABECALHO = {"apikey": ANON_KEY, "Authorization": f"Bearer {ANON_KEY}",
              "Content-Type": "application/json"}

fila_vel = queue.Queue() # (momento, velocidade)
fila_tag = queue.Queue() # (momento, epc)
parar = threading.Event()

def log(*a):
    print(datetime.now().strftime("%H:%M:%S"), *a, flush=True)

# --- Thread 1: le o radar -----
def ler_radar():
    while not parar.is_set():
        try:
            r = serial.Serial(PORTA, BAUD, timeout=1)
            time.sleep(0.5)
            # Configuracao validada em bancada no OPS243-C-FC:
            # Od desliga o reporte de distancia (senao inunda a porta)
            # OS liga o reporte de velocidade
            # OM liga a magnitude do sinal (usada para filtrar ruido)
            # UK unidade em km/h
            # A configuracao NAO persiste: o radar volta ao padrao a cada
            # reconexao, por isso e reenviada aqui toda vez.
            #
            comandos = [b"Od\n", b"OS\n", b"OM\n", b"UK\n"]

            # "M>" define SpeedMagnitudeMin: a forca minima do eco, NAO a
            # velocidade minima que o nome sugere. Alimentado com MAG_MIN,
            # faz o radar descartar o ruido de fundo na origem -- em vez de
            # transmitir milhares de linhas inuteis para o Pi jogar fora.
            # A filtragem no codigo continua como segunda barreira.
            if MAG_MIN > 0:
                comandos.append(f"M>{MAG_MIN:.0f}\n".encode())

            for cmd in comandos:
                r.write(cmd); time.sleep(0.3)
            log("Radar conectado em", PORTA,
              f"(magnitude minima no radar: {MAG_MIN:.0f})" if MAG_MIN > 0
              else "(sem filtro de magnitude)")

            while not parar.is_set():
                linha = r.readline().decode(errors="ignore").strip()
                if not linha:
                    continue
                stats["linhas"] += 1
                if DEBUG:
                    log(" radar:", linha)

                v, mag = extrair_leitura(linha)
                if v is None:
                    continue
                stats["leituras"] += 1

```

continua na próxima página — parte 2 de 7

```

        if abs(v) < VEL_MIN:
            stats["descartadas"] += 1
            if DEBUG:
                log(f"    descartada: {abs(v):.1f} < VELOCIDADE_MIN={VEL_MIN}")
            continue
        # Descarta eco fraco demais para ser veículo (ruído/fantasma)
        if MAG_MIN > 0 and mag is not None and mag < MAG_MIN:
            stats["descartadas"] += 1
            if DEBUG:
                log(f"    descartada: magnitude {mag} < MAGNITUDE_MIN={MAG_MIN}")
            continue
        fila_vel.put((time.time(), abs(v)))
    except Exception as e:
        log("Radar caiu:", e, "- retentando em 5s")
        time.sleep(5)

def extrair_leitura(linha):
    """
    Formatos do OPS243-C (confirmados em bancada):

        "kmph",2.6                sem magnitude    (OM desligado)
        "kmph",171,2.6            com magnitude    (OM ligado)
        "m",2.0                   distancia        -> ignorada
        {"SpeedUnit":"kmph"}      resposta de comando -> ignorada

    Com OM ligado a velocidade é sempre o ÚLTIMO campo, e a magnitude
    vem antes dela. A magnitude é o que separa alvo real de fantasma.

    Retorna (velocidade_kmh, magnitude). Magnitude é None quando o
    radar não está reportando. (None, None) quando não é velocidade.
    """
    linha = linha.strip()
    if not linha or linha[0] != ' ':
        return None, None                # resposta de comando ou lixo

    partes = linha.split(",")
    unidade = partes[0].strip(' ').lower()
    if unidade not in ("kmph", "kmh", "mps"):
        return None, None                # "m" = distancia

    try:
        if len(partes) >= 3:
            mag, vel = float(partes[1]), float(partes[2])
        elif len(partes) == 2:
            mag, vel = None, float(partes[1])
        else:
            return None, None
    except ValueError:
        return None, None

    if unidade == "mps":
        vel *= 3.6                        # m/s -> km/h
    return vel, mag

```

continua na próxima página — parte 3 de 7

```
# --- Thread 2: le as tags UHF -----
def ler_tags():
    """
    Recebe as leituras do Control ID IDUHF pelo modo Monitor.

    O leitor faz POST para <IP do Pi>:<LEITOR_PORTA>/api/notifications/...
    a cada evento. Configuracao no proprio leitor, em
    Configuracoes -> Modo de Operacao -> Configurar modo monitor.

    Formato observado no equipamento (firmware V5.19.2):

    POST /api/notifications/dao
    {"object_changes": [{"object": "access_logs", "type": "inserted",
        "values": {"card_value": "223338326031", "user_id": "", ...}}]}

    POST /api/notifications/device_is_alive
    {"access_logs": 3, "device_id": ..., "time": ...}

    A tag vem em card_value. Deixamos o leitor SEM as tags cadastradas de
    proposito: quem resolve tag -> veiculo e o nosso banco, na tabela
    sms_veiculos_rfid. Assim o cadastro nao fica duplicado em dois lugares.

    Usamos o modo Monitor e nao o modo Online porque o Monitor e
    assincrono: se o Pi cair, o leitor continua operando normalmente.
    No modo Online ele ficaria esperando resposta a cada leitura.
    """
    from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

    class Receptor(BaseHTTPRequestHandler):
        protocol_version = "HTTP/1.1"

        def do_POST(self):
            tamanho = int(self.headers.get("Content-Length") or 0)
            bruto = self.rfile.read(tamanho) if tamanho else b""
            self._responder() # responde antes de processar
            try:
                dados = json.loads(bruto.decode("utf-8", "replace"))
            except Exception:
                return

            if self.path.endswith("/device_is_alive"):
                stats["leitor_visto_em"] = time.time()
                return

            agora = time.time()
            for mudanca in dados.get("object_changes", []):
                if mudanca.get("object") != "access_logs":
                    continue
                if mudanca.get("type") != "inserted":
                    continue
                v = mudanca.get("values", {}) or {}
                tag = (v.get("card_value") or v.get("uhf_tag") or "").strip()
                if not tag or tag in ("0", ""):
                    continue
                fila_tag.put((agora, tag.upper()))
                stats["tags"] += 1
                if DEBUG:
                    log(f" tag: {tag}")
```

continua na próxima página — parte 4 de 7

```

def _responder(self):
    # event 6 = negado. Nao ha portao para abrir aqui; so registramos.
    corpo = json.dumps({"result": {"event": 6, "user_id": 0,
                                   "user_name": "", "portal_id": 1,
                                   "actions": []}}).encode()

    self.send_response(200)
    self.send_header("Content-Type", "application/json")
    self.send_header("Content-Length", str(len(corpo)))
    self.end_headers()
    self.wfile.write(corpo)

def do_GET(self):
    self._responder()

def log_message(self, *a):
    pass # o log e nosso, nao o do http.server

while not parar.is_set():
    try:
        srv = ThreadingHTTPServer(("0.0.0.0", LEITOR_PORTA), Receptor)
        srv.timeout = 1
        log(f"Aguardando o leitor UHF na porta {LEITOR_PORTA}")
        while not parar.is_set():
            srv.handle_request()
        srv.server_close()
    except Exception as e:
        log("Servidor do leitor caiu:", e, "- retentando em 5s")
        time.sleep(5)

# --- Envio ao sistema -----
def enviar(velocidade, epc, momento):
    corpo = {
        "p_device_token": TOKEN,
        "p_tag_epc": epc or "",
        "p_velocidade_kmh": round(velocidade, 1),
        "p_sentido": "indefinido",
        "p_foto_url": None,
        "p_detectado_em": datetime.fromtimestamp(
            momento, tz=tz=timezone.utc).isoformat(),
    }
    try:
        resp = requests.post(RPC, headers=CABECALHO, json=corpo, timeout=10)
        resp.raise_for_status()
        r = resp.json()
        if r.get("infração"):
            log(f"INFRAÇÃO {velocidade:.0f} km/h tag={epc} "
                f"gravidade={r.get('gravidade')}")
        else:
            log(f"passagem {velocidade:.0f} km/h tag={epc or '-'}")
    except Exception as e:
        log("Falha ao enviar:", e)
        gravar_pendente(corpo)

```

continua na próxima página — parte 5 de 7


```

def gravar_pendente(corpo):
    """Sem internet: guarda em disco para reenviar depois."""
    with open("pendentes.jsonl", "a") as f:
        f.write(json.dumps(corpo) + "\n")

def reenviar_pendentes():
    if not os.path.exists("pendentes.jsonl"):
        return
    linhas = [l for l in open("pendentes.jsonl") if l.strip()]
    if not linhas:
        return
    log(f"Reenviando {len(linhas)} registro(s) pendente(s)")
    restantes = []
    for ln in linhas:
        try:
            resp = requests.post(RPC, headers=CABECALHO,
                                json=json.loads(ln), timeout=10)
            resp.raise_for_status()
        except Exception:
            restantes.append(ln)
    with open("pendentes.jsonl", "w") as f:
        f.writelines(restantes)

# --- Correlacionador: junta velocidade + tag -----
def correlacionar():
    pico = 0.0          # maior velocidade da passagem atual
    inicio = None       # quando a passagem começou
    ultima = 0.0        # ultima leitura do radar
    tags = []           # tags vistas recentemente
    ultimo_reenvio = time.time()

    while not parar.is_set():
        agora = time.time()

        # Coleta tudo que chegou
        while not fila_vel.empty():
            t, v = fila_vel.get()
            if v > pico:
                pico = v
            if inicio is None:
                inicio = t
            ultima = t

        while not fila_tag.empty():
            tags.append(fila_tag.get())

        # Descarta tags velhas demais para pertencer a esta passagem
        tags = [(t, e) for (t, e) in tags if agora - t < JANELA * 3]

```

continua na próxima página — parte 6 de 7

```

# Fecha a passagem quando o radar silencia OU quando ela ja dura
# tempo demais. A segunda condicao e a rede de segurança: sem ela,
# ruído contínuo impede o silêncio e nada é enviado nunca.
if inicio is not None:
    silenciou = (agora - ultima) > FIM_PASS
    estourou = (agora - inicio) > DUR_MAX
    if silenciou or estourou:
        candidatas = [(abs(t - inicio), e) for (t, e) in tags
                        if abs(t - inicio) <= JANELA]
        epc = min(candidatas)[1] if candidatas else None
        if estourou and not silenciou:
            log(f"[aviso] passagem fechada por tempo ({DUR_MAX:.0f}s) - "
                "sinal contínuo. Suspeita de ruído: use MAGNITUDE_MIN")
        stats["passagens"] += 1
        enviar(pico, epc, inicio)
        if epc:
            tags = [(t, e) for (t, e) in tags if e != epc]
        pico, inicio, ultima = 0.0, None, agora

# Tenta reenviar o que ficou preso a cada 60s
if agora - ultimo_reenvio > 60:
    reenviar_pendentes()
    # Heartbeat: prova que o script esta vivo e mostra onde os
    # dados estao parando (radar mudo? filtro comendo tudo?)
    log(f"[status] {stats['linhas']} linhas do radar, "
        f"{stats['leituras']} leituras validas, "
        f"{stats['descartadas']} descartadas pelo filtro, "
        f"{stats['passagens']} passagens enviadas, "
        f"{stats['tags']} tags lidas")
    if stats["linhas"] == 0:
        log("[status] radar nao emitiu nada - confira a config e o cabo")
    if LEITOR_PORTA:
        visto = stats["leitor_visto_em"]
        if not visto:
            log("[status] leitor UHF nunca chamou - confira o modo "
                "monitor na tela dele (hostname, porta e endpoint)")
        elif agora - visto > 120:
            log(f"[status] leitor UHF sem contato ha "
                f"{int(agora - visto)}s")
        ultimo_reenvio = agora

time.sleep(0.05)

# --- Inicio -----
if __name__ == "__main__":
    log("Checkpoint iniciando...")
    threading.Thread(target=ler_radar, daemon=True).start()
    if LEITOR_PORTA:
        threading.Thread(target=ler_tags, daemon=True).start()
    else:
        log("AVISO: leitor UHF não configurado - só velocidade sera enviada")
    try:
        correlacionar()
    except KeyboardInterrupt:
        parar.set()
        log("Encerrado.")

```

Três detalhes que valem entender

Pico de velocidade: o radar dispara várias leituras por passagem. O script guarda a maior, que é a que interessa.

Funciona sem internet: se a conexão cair, os registros vão para o arquivo `pendentes.jsonl` e são reenviados sozinhos quando voltar. Nada se perde.

Tag opcional: se nenhuma etiqueta for lida, a passagem é registrada mesmo assim, com o veículo em branco — útil para detectar visitante ou etiqueta descolada.

6 Deixar rodando sozinho

Configurar como serviço faz o programa iniciar junto com o Pi e reiniciar sozinho se travar. Sem isso, uma queda de energia deixa o ponto morto até alguém subir no poste.

6.1 Criar o serviço

```
sudo nano /etc/systemd/system/checkpoint.service
```

```
[Unit]
Description=Checkpoint de velocidade - Sistema Apice
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=SEU_USUARIO
WorkingDirectory=/home/SEU_USUARIO/checkpoint
ExecStart=/home/SEU_USUARIO/checkpoint/.venv/bin/python checkpoint.py
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Restart=always é a linha que garante o retorno automático após qualquer falha.

Troque SEU_USUARIO pelo seu login no Pi (o que aparece no prompt do terminal). Se estiver errado, o serviço sobe e morre em seguida, sem mensagem óbvia.

O que acontece quando falta energia

O Raspberry Pi não tem botão de liga: energia presente significa ligado. Então a sequência é automática — energia volta, o Pi sobe, o systemd inicia o serviço, o script reconecta no radar e **reenvia toda a configuração**. Em cerca de um minuto o ponto está operando de novo, sem ninguém ir até lá. É aqui que a decisão de não gravar com A! compensa: não sobra estado escondido na flash do radar.

O risco que o systemd não cobre

Desligamento abrupto corrompe cartão SD — é a causa número um de Raspberry Pi morto em campo, e nenhuma configuração de serviço protege contra isso. A proteção real é a bateria prevista no projeto: com ela no meio, o Pi nunca vê queda, só descarga lenta. Em ponto ligado direto na rede elétrica, use nobreak pequeno ou cartão de grau industrial.

6.2 Ativar

```
sudo systemctl daemon-reload
sudo systemctl enable checkpoint
sudo systemctl start checkpoint

sudo systemctl status checkpoint
```

6.3 Comandos do dia a dia

Para que serve	Comando
Ver o que esta acontecendo agora	<code>sudo journalctl -u checkpoint -f</code>
Ver as últimas 100 linhas	<code>sudo journalctl -u checkpoint -n 100</code>
Reiniciar após mudar configuração	<code>sudo systemctl restart checkpoint</code>
Parar para manutenção	<code>sudo systemctl stop checkpoint</code>
Conferir registros presos sem internet	<code>wc -l ~/checkpoint/pendentes.jsonl</code>

7 Validar antes de liberar

Não anuncie o checkpoint para a equipe antes de passar por estes quatro testes. Radar mal calibrado gera contestação e desgaste.

Teste 1 — O sistema recebe?

Sem hardware nenhum, do seu próprio computador, simule uma passagem. Se aparecer na tela, o caminho até o banco está certo.

```
curl -X POST \
  "https://dadosfrota.apicesystem.shop/rest/v1/rpc/registrar_passagem_checkpoint" \
  -H "apikey: <chave anon>" \
  -H "Authorization: Bearer <chave anon>" \
  -H "Content-Type: application/json" \
  -d '{
    "p_device_token": "<seu token>",
    "p_tag_epc": "TESTE123",
    "p_velocidade_kmh": 62,
    "p_sentido": "entrada"
  }'
```

Resposta esperada: {"ok": true, "infração": true, "gravidade": "media", ...}

Teste 2 — Velocidade confere?

Peça para alguém passar de carro em velocidade constante conhecida (30, 40 e 50 km/h), conferindo pelo velocímetro. Compare com o que aparece na aba **Passagens**. Diferença de 1 a 2 km/h é normal; acima disso, revise o ângulo do radar — ele mede a velocidade na direção em que aponta, então quanto mais de lado, mais subestima.

Teste 3 — A tag é reconhecida?

Passe um veículo **com etiqueta cadastrada**. Na aba Passagens deve aparecer a placa, não *não identificado*. Se aparecer sem placa:

- Confira se o EPC foi cadastrado exatamente igual, sem espaços
- Aumente a janela de correlação para 3.0 no arquivo .env
- Verifique se antena e radar cobrem o mesmo trecho da via

Teste 4 — A infração nasce sozinha?

Passe deliberadamente acima do limite mais a tolerância. A infração deve aparecer na aba **Infrações** em segundos, com a gravidade calculada.

Quanto acima do limite	Gravidade atribuída
até 20%	Leve
de 20% a 50%	Média
de 50% a 80%	Grave
acima de 80%	Gravíssima

Rode uma semana em silêncio

Antes de comunicar a equipe, deixe o ponto coletando sem notificar ninguém. Isso mostra o comportamento real da via, revela falso positivo e dá base para você ajustar o limite e a tolerância com dado, não com achismo.

Quando algo não funciona

Sintomas mais comuns e o que checar primeiro, em ordem.

Sintoma	Causa provável	O que fazer
/dev/ttyACM0 não existe	Cabo USB só de carga, ou radar sem energia	Troque por cabo de dados; confira o LED do radar
Permission denied na porta serial	Usuário fora do grupo dialout	sudo usermod -aG dialout \$USER e reinicie
Radar responde mas velocidade vem estranha	Unidade em mph, não km/h	Reenvie UK e depois A! para gravar
device_token invalido	Token errado, ou checkpoint desativado na tela	Recopie o token; confirme que o ponto esta ativo
Passagem registra sem placa	Tag não cadastrada, fora de alcance ou janela curta	Confira o EPC; aumente JANELA_S; realinhe a antena
Velocidade sempre menor que a real	Radar muito de lado em relação a via	Reduza o ângulo — quanto mais de frente, melhor
Mesma passagem registrada duas vezes	FIM_PASS curto para veículo longo	Aumente FIM_PASS de 1.2 para 2.0 no script
Nada chega ao sistema, sem erro no log	Sem internet no ponto	Veja pendentes.jsonl; teste com ping 8.8.8.8
Serviço morre e não volta	Erro de configuração no .env, ou usuário errado na unit	sudo journalctl -u checkpoint -n 50
Script roda mas nada é enviado, terminal mudo	Ruído contínuo mantém a passagem sempre aberta	Ligue DEBUG=1 e calibre MAGNITUDE_MIN (Etapa 3.6)
Log mostra 'passagem fechada por tempo'	Sinal contínuo passando pelo filtro	Suba o MAGNITUDE_MIN e confira a mira do radar
Leituras com velocidade absurda (acima de 100)	Ruído de fundo, sempre com magnitude baixa	MAGNITUDE_MIN corta; o alvo real tem magnitude alta
Radar volta a reportar distância sozinho	Configuração não persiste e ele foi religado	Normal: o script reenvia os comandos ao conectar
Passagem sempre sai como veículo não identificado	Tag cadastrada no formato da etiqueta, sem converter	Compare o valor salvo com o do log; use o seletor de formato
Leitor UHF nunca chamou o Pi	Hostname, porta ou endpoint errados no modo monitor	Confira a tela do leitor e se o Pi tem IP fixo
Leitor parou de chamar depois de um reboot	O Pi trocou de IP e o leitor fala sozinho	Reserve IP fixo do Pi no roteador por MAC
Tag lida mas passagem sai sem ela	Leitor e radar cobrem trechos diferentes da via	Alinhe os dois; aumente JANELA_S para 3.0

Instalação no poste

Recomendações para o ponto sobreviver a chuva, vento e falta de energia.

Energia

Um Raspberry Pi 4 com radar e leitor consome em torno de 15 a 25 W. Para ponto sem rede elétrica, dimensione com folga de dois dias sem sol — chuva prolongada é o que costuma derrubar instalação subdimensionada.

- Paine solar de 100 W com controlador de carga
- Bateria estacionária de 12 V, 100 Ah
- Conversor 12 V para 5 V de boa qualidade — fonte ruim causa reinício aleatório, o defeito mais difícil de diagnosticar

Proteção

- Caixa hermética IP65 ou IP67, com prensa-cabo em toda entrada
- Saco de sílica dentro da caixa contra condensação
- Radar e antena UHF **fora** da caixa metálica — metal bloqueia tanto o sinal do radar quanto o de RF do leitor
- Altura de 2,5 a 3 m: acima do alcance de vandalismo e fora do respingo de lama

Rede

- Modem 4G com plano de dados — o volume é baixo, poucos MB por mês
- Se houver Wi-Fi da obra no alcance, use como principal e deixe o 4G reserva
- O script guarda tudo em disco quando cai, então rede instável não perde registro

Antes de fechar a caixa

Confirme que você consegue acessar o Pi por SSH de fora e que `systemctl status checkpoint` mostra `active (running)`. Descobrir problema com a caixa lacrada a 3 metros do chão custa caro.

Referência: a chamada ao sistema

Contrato da função que o Pi chama. Útil para integrar outro equipamento no futuro.

Endpoint

```
POST https://dadosfrota.apicesystem.shop
/rest/v1/rpc/registrar_passagem_checkpoint
```

Cabeçalhos:

```
apikey: <chave anon>
Authorization: Bearer <chave anon>
Content-Type: application/json
```

Parâmetros

Campo	Tipo	Obrigatório	Descrição
p_device_token	texto	sim	Token do checkpoint, copiado da tela
p_tag_epc	texto	sim	EPC lido. Vazio se não identificou
p_velocidade_kmh	número	sim	Velocidade medida em km/h
p_sentido	texto	não	entrada, saída ou indefinido
p_foto_url	texto	não	URL da foto de evidência
p_detectado_em	data/hora	não	Momento da passagem. Padrão: agora

Resposta

```
{
  "ok": true,
  "passagem_id": "a3f1...",
  "veiculo_conhecido": true,
  "limite_kmh": 40,
  "infração": true,
  "gravidade": "media"
}
```

O que o servidor faz com isso

Autentica o token, procura o veículo pela tag, aplica o limite vigente do ponto e grava a passagem. Um gatilho no banco avalia se passou do limite mais a tolerância e, se passou, cria a infração com a gravidade calculada. O dispositivo não precisa saber de nenhuma dessas regras.

Do lado do sistema

Com o ponto rodando, a equipe de SMS trabalha assim:

- **Aba Checkpoints** mostra online ou offline por ponto — se ficar mais de 15 minutos sem contato, aparece offline
- **Aba Passagens** lista tudo, dentro e fora do limite

- **Aba Infrações** traz só quem excedeu, com botão **Gerar desvio SMS** para escalar ao processo formal de tratativa
- Enquanto o hardware não chega, **Registro manual** permite operar com radar portátil pelas mesmas regras